

20-52-Q5 -b

Q: (a) CNN

(b) K-means

20-S2-Q5

5 (a) Convolutional Neural Networks (CNNs)

High-level idea

A CNN is a deep-learning model that learns a hierarchy of spatial features from grid-like inputs (e.g., images, spectrograms). It does so with three architectural principles:

Principle	What it does	Why it matters
Local receptive fields	Each neuron “looks” at only a small neighborhood of the previous layer.	Detects elementary patterns (edges, corners) that are reused across the image.
Weight sharing (convolution kernels)	The same kernel slides over the entire feature map.	Gives translation equivariance and reduces parameters dramatically.
Pooling / subsampling	Down-samples each feature map (e.g., max- or average-pool).	Introduces local translation invariance and shrinks spatial resolution, enabling deeper stacks.

Deep learning (including CNNs) sits inside the broader neural-network family within machine learning [cite turn2file14](#).

Typical layer stack

1	Input (H×W×C)
2	└ Conv(k=3×3, 32 filters) + ReLU
3	└ Conv(k=3×3, 32) + ReLU
4	└ MaxPool(2×2)
5	└ Conv(k=3×3, 64) + ReLU
6	└ ... (repeat)
7	└ Global-Average-Pool
8	└ Fully-connected → Soft-max

- Modern variants add batch-normalisation, dropout, residual/skip connections, dilated or depth-wise separable convolutions, etc.*

Invariances and equivariances

Mechanism	Gives you ...	Still need ...
Convolution + weight sharing	Translation equivariance (if a pattern moves by Δx , its activation map moves by the same Δx).	Pooling or global average to turn equivariance into invariance.

Mechanism	Gives you ...	Still need ...
Pooling (max/avg)	Local translation invariance and slight robustness to small scale/rotation.	Data augmentation or specialised layers for strong scale / rotation invariance.

Main training loop (mini-batch SGD)

1. Forward pass

Compute activations layer-by-layer with current weights.

2. Loss computation

Compare predictions to labels (e.g., cross-entropy).

3. Backward pass

Back-propagate gradients through:

- Fully-connected layers (as in standard MLP)
- Pooling layers (routing indices for max-pool)
- Convolution layers (gradients w.r.t. kernels obtained by another convolution)

4. Parameter update

$\theta \leftarrow \theta - \eta \nabla_{\theta} L$ using SGD/Adam, optionally with momentum, weight decay, learning-rate schedules.

5. Regularisation & monitoring

Data augmentation, dropout, early stopping, validation accuracy.

The loop repeats for many epochs until the validation metric plateaus or a stopping criterion is met.

Solution (b)

① $k=3$

② Sample 1, 2, 3 as centroid C_1, C_2, C_3

③ Iteration 1

Assign all data points to centroid

distant Sample	C_1	C_2	C_3	Assign
1	0	1	1	C_1
2	1	0	1.41	C_2
3	1	1.41	0	C_3
4	6.08	6	5.10	C_3
5	7	7.07	6	C_3
6	6	6.08	5	C_3
7	7.07	6.08	7	C_2
8	6	5	6.08	C_2
9	6.08	5.10	6	C_2

$C_1 : \{1\}$

$C_2 : \{2, 7, 8, 9\}$

$C_3 : \{3, 4, 5, 6\}$

④ New centroid

$$C_1 \rightarrow (0,0)$$

$$C_2 \rightarrow \left(\frac{0+1+0+1}{4}, \frac{1+7+6+6}{4} \right) = (0.5, 5)$$

$$C_3 \rightarrow \left(\frac{1+6+7+6}{4}, \frac{1}{4} \right) = (5, 0.25)$$

⑤ Iteration 2

distance Sample	C1	C2	C3	Assign
1	0	5.02	5.01	C1
2	1	4.03	5.06	C1
3	1	5.02	4.01	C1
4	6.08	6.80	1.25	C3
5	7	8.20	2.02	C3
6	6	7.43	1.03	C3
7	7.07	2.06	7.85	C2
8	6	1.12	7.62	C2
9	6.08	1.12	7.01	C2

$$C_1 = \{1, 2, 3\}$$

$$C_2 = \{7, 8, 9\}$$

$$C_3 = \{4, 5, 6\}$$

$$⑥ C_1 \rightarrow (0.3333, 0.3333)$$

$$C_2 \rightarrow (0.6667, 6.3333)$$

$$C_3 \rightarrow (6.3333, 0.3333)$$

⑦ Iteration 3

distance Sample	C1	C2	C3	Assign
1	0.47	6.37	6.34	C1
2	0.75	5.37	6.37	C1
3	0.75	6.34	5.34	C1
4	5.71	7.54	0.75	C3
5	6.67	8.96	0.75	C3
6	5.68	8.28	0.47	C3
7	6.70	0.75	8.54	C2
8	5.68	0.75	8.50	C2
9	5.71	0.47	7.78	C2

$$C_1 = \{1, 2, 3\}$$

$$C_2 = \{7, 8, 9\}$$

$$C_3 = \{4, 5, 6\}$$

⑧ $C_1 \rightarrow (0.3333, 0.3333)$

$C_2 \rightarrow (0.6667, 6.3333)$

$C_3 \rightarrow (6.3333, 0.3333)$

⑨ There is no change in the centroids
and the assignment of samples